

JALabs_SymSearch — User & Operations Guide

A practical guide for administrators and operators: what the module does, how to configure it, how indexing works, how to run and monitor it, and how to troubleshoot. For architecture and developer internals see [TECHNICAL.md](#).

1. What this module does

Antoine Online's catalog search runs on **Mirasvit Search Ultimate** over **OpenSearch**, which matches on keywords (BM25). SymSearch adds a second, *meaning-based* signal on top of it:

- A customer typing "**gift for a 7 year old who loves space**" gets a telescope and astronomy books — even though no product title contains those words.
- A **zero-result** query ("self improvement habit building") now returns *Tiny Habits*, *Badass Habits*, etc.
- A French or Arabic query against English product data ("livre de cuisine libanaise", "كتب تطوير الذات") returns the right books.

It does this by storing a **vector embedding** for every product and, at search time, combining the keyword score with the vector (semantic) score in a single OpenSearch *hybrid* query. Keyword relevance and all of Mirasvit's tuning stay exactly as they were; semantic is an additive signal.

Key principle — it never makes search worse. If the embedding service is down, slow, or a product hasn't been embedded yet, search silently falls back to the normal keyword behaviour. Disabling the module returns you to stock Mirasvit.

Scope (v1): the storefront **search results page** only. Autocomplete, GraphQL/headless search, and category browse are intentionally untouched.

2. Admin configuration

Stores → Configuration → JALabs → Semantic Search. Five groups:

General

Field	Default	Meaning
Enable Module (indexing infrastructure)	No	Master switch (default scope). Turns on the vector field in the index and the embedding pipeline. Required for everything else.
Enable Hybrid Search on Storefront	No	Per store view . Turns on the semantic query rewrite for that view. Requires the module enabled <i>and</i> embeddings generated.
Debug Logging	No	Verbose <code>[symsearch]</code> lines in <code>var/log/system.log</code> . Turn off in production.

The two-switch design lets you generate embeddings and reindex with the module **enabled** while the storefront still serves pure keyword results, then flip **storefront** on per store view only when you're ready.

Embedding Provider

Field	Default	Meaning
Provider	OpenAI	<code>OpenAI</code> , <code>Google Gemini</code> , or <code>Voyage AI</code> . (Anthropic has no embeddings API; Voyage is the Anthropic-recommended option.)
API Key	—	Stored encrypted . Never logged.
Model	text-embedding-3-small	<code>openai</code> : <code>text-embedding-3-small</code> · <code>gemini</code> : <code>gemini-embedding-001</code> · <code>voyage</code> : <code>voyage-3.5-lite</code> (or <code>voyage-4</code>).
Dimensions	512	Vector size. Changing this requires a full re-embed + reindex (see §7). 512 is a good quality/cost/RAM balance.

Indexing

Field	Default	Meaning
Attributes To Embed	name,short_description,description	Comma-separated product attribute codes. Add e.g. <code>manufacturer</code> , author attributes, or category text to enrich the embedded text. Select/dropdown attributes are embedded by their label , not the option ID.
API Batch Size	96	Texts per provider call.
API Throttle (ms between calls)	2000	Pause between provider calls during bulk generation. Protects against per-minute rate limits. See §6.3.

Ranking

Field	Default	Scope	Meaning
Keyword Weight	0.7	Default only*	Weight of the BM25 sub-query in the blended score.
Semantic Weight	0.3	Default only*	Weight of the vector sub-query.
k (nearest neighbours)	100	Per store view	How many vector neighbours to retrieve per query.
Min Semantic Score	0	Per store view	<code>0</code> = use <code>k</code> . If set <code>>0</code> , retrieves by score floor instead of <code>k</code> (mutually exclusive — never both).
Query Embedding Timeout (ms)	800	Per store view	Budget for embedding the live query. Exceeded → keyword fallback. Floor 100ms.

* Weights drive a single, cluster-global OpenSearch pipeline, so they are default-scope only. **After changing either weight you must run `symsearch:pipeLine:setup`** (§4) for it to take effect.

3. How indexing / embedding works (the lifecycle)

You don't embed products by hand — the module keeps them in sync. Each product/store-view has a row with a **status**:

```
pending → queued → ready
           ↘ failed ↗ (retried by the nightly sweep)
```

1. **Becomes stale (pending)** — when a product is saved (observer), or when the nightly sweep notices its `updated_at` changed (catches imports / API writes that bypass the save event), or when you run `embed:generate`.
2. **Dispatched (queued)** — the dispatch cron (every 5 min) or `embed:generate` claims a batch and publishes it to RabbitMQ.
3. **Embedded (ready)** — the queue consumer builds each product's text from the configured attributes, hashes it, and (only if that exact text hasn't been embedded before for the current model) calls the provider, stores the vector, and marks the row ready.
4. **Injected into the search index** — the **next catalog search reindex** reads the ready vectors and writes them into the OpenSearch product documents. **Reindex never calls the embedding API** — it only reads stored vectors.

Two cost-savers worth knowing: - **Content-hash dedup**: vectors are keyed by `sha1(text) + model`. Identical text across store views (e.g. an untranslated English title shown in the French view) is embedded **once**. On Antoine's catalog this cut ~889k product/store rows down to ~358k actual API calls. - **No-op reindex cost**: because vectors live in the database, you can reindex as often as you like with zero embedding cost.

4. Command-line tools

Run via `bin/magento` (in this environment: `warden env exec -T php-fpm php bin/magento ...`).

Command	What it does
<code>symsearch:embed:generate [--store=ID] [--limit=N] [--force]</code>	Seeds rows for products that have none, then queues <code>pending</code> items for embedding. <code>--force</code> resets all rows to pending first (use for a full re-embed). <code>--store</code> limits to one store id; <code>--limit</code> caps how many to queue.
<code>symsearch:embed:status</code>	Per-store table: pending / queued / ready / failed / coverage %, plus the active model version. Your main monitoring command.
<code>symsearch:pipeline:setup</code>	Creates/updates the OpenSearch normalization pipeline from the configured weights, and verifies the <code>opensearch-knn</code> and <code>opensearch-neural-search</code> plugins are present. Run after install and after any weight change.
<code>symsearch:relevance:run <csv> [--store=ID] [--suggest]</code>	Runs a CSV of test queries through real search. Without <code>--suggest</code> it asserts (exits non-zero on failure) — your regression gate. With <code>--suggest</code> it prints the current top-5 SKUs per query, for curating the expected answers.

Running the consumer. Embedding happens in the queue consumer `jalabsSymsearchEmbed`. In production Magento's `consumers_runner` cron starts it automatically. For a manual/bulk run:

```
bin/magento queue:consumers:start jalabsSymsearchEmbed --max-messages=1800
```

(It exits after `--max-messages` messages; loop it, or run several for a big backfill. Watch the rate limit — see §6.3.)

5. Admin panel (Operations)

The **Operations** tab at **Stores** → **Configuration** → **JALabs** → **Semantic Search** → **Operations** is the admin-UI equivalent of the CLI commands (§4). Use it when you prefer not to drop into a shell.

Coverage table — a row per store view showing pending / queued / ready / failed counts and coverage %, plus the active model version and a pipeline/plugin health indicator.

Buttons — all four are asynchronous: they update database state and return immediately; the dispatch cron (every 5 min) and the queue consumer do the actual embedding work.

Button	What it does
Refresh status	Reloads the coverage table from the database.
Generate embeddings	Seeds rows for products that have none, then queues all <code>pending</code> items — the gap-fill equivalent of <code>symsearch:embed:generate</code> .
Force re-embed	Resets all products to <code>pending</code> and invalidates the <code>catalogsearch_fulltext</code> index. Prompts for confirmation (this is a paid full re-embed). Equivalent to <code>symsearch:embed:generate --force</code> . Reindex once coverage recovers.
Refresh / verify pipeline	Creates or updates the OpenSearch normalization pipeline from the current weights, and verifies the <code>opensearch-knn</code> / <code>opensearch-neural-search</code> plugins. Equivalent to <code>symsearch:pipeline:setup</code> .

Staleness banner — when embedding-affecting settings (Attributes To Embed, Model, or Dimensions) change after a successful embed run, a persistent admin banner appears prompting a Force re-embed. It clears once a Force re-embed completes (or after the first Generate on a fresh install). Both the panel's Force re-embed and `symsearch:embed:generate --force` (CLI) re-establish the drift baseline and clear the banner. The same signal is exposed as the stale indicator in `symsearch:embed:status` .

Queue and cron must be running. The buttons only schedule work — nothing is embedded synchronously during the request. The consumer (`queue:consumers:start jalabsSymsearchEmbed`) and the dispatch cron must be active to process the queued items.

6. First-time setup & backfill

1. **Configure** the provider, API key, model, dimensions (\$2). Set **Enable Module = Yes**. Leave **Storefront = No** for now.
2. **Create the pipeline:** `bin/magento symsearch:pipeline:setup` (confirms the OpenSearch plugins and creates `jalabs_symsearch_hybrid`).
3. **Seed & queue:** `bin/magento symsearch:embed:generate`
4. **Run the consumer** until coverage is high: `bin/magento queue:consumers:start jalabsSymsearchEmbed --max-messages=2000` Monitor with `bin/magento symsearch:embed:status` .
5. **Reindex** to inject vectors: `bin/magento indexer:reindex catalogsearch_fulltext`
6. **Validate** with `symsearch:relevance:run ... --suggest` , then enable **Storefront = Yes** on **one** store view and compare Mirasvit's zero-result rate over a week before expanding.

6.1 Cost

Embedding the full catalog is a **one-time** cost of a few dollars at 512-dim (content-hash dedup means you pay per *unique* text, not per product×store). Steady-state cost is tiny (only changed products, and repeat live queries are Redis-cached for 14 days).

6.2 Backfill scale

A ~300k-product catalog × 3 store views backfills in a few hours on a single throttled consumer. EN/FR/AR views finish much faster than the first view if their product text is shared (dedup). Run the consumer in a dedicated shell or under supervisor; it's safe to run several in parallel **as long as** the combined rate stays under the provider's per-minute limit.

6.3 Rate limits (important for Gemini)

Embedding providers limit **requests per minute**, and some count **each text in a batch as a separate request**. Gemini's paid tier is ~3,000 requests/min, and a batch of 96 texts = 96 requests — so an unthrottled consumer hits the ceiling in seconds.

The module handles this two ways: - **Proactive throttle** — `API Throttle (ms between calls)` (default 2000ms). With batch 96, 1000–2000ms keeps a single consumer comfortably under Gemini's limit (~2,800–5,700 texts/min). Lower it for OpenAI/Voyage (higher limits); raise it if you see 429s. - **Reactive backoff** — a 429 is caught, the provider's "retry after" hint is parsed, and the batch is retried (up to 8 times) instead of failing. Transient network blips are also retried (up to 3 times). So short rate-limit or connectivity hiccups self-heal; they don't burn items to `failed`.

If you do see `failed` items pile up, just re-run `embed:generate` (or wait for the nightly sweep) — dedup makes the retry nearly free.

7. Day-2 operations

Changing which attributes are embedded: update *Attributes To Embed*, then `symsearch:embed:generate --force` (re-hashes everyone), run the consumer, then reindex.

Changing the model or dimensions: the model version changes, which invalidates every stored vector. Run `symsearch:embed:generate --force`, the consumer, then a full reindex (the `knn_vector` mapping picks up the new dimension on rebuild). Budget for a full re-embed cost.

Changing ranking weights: edit the weights, run `symsearch:pipeline:setup`, flush cache. No reindex needed — weighting happens at query time.

Tuning relevance: keep a curated `dev/symsearch/relevance-suite.csv` (see §8). Adjust weights / `k`, re-run the suite, compare. More keyword weight = closer to stock behaviour; more semantic weight = more "fuzzy intent" matching.

Rollback: set **Storefront = No** (instant, no reindex). Set **Module = No** to also stop the indexing pipeline. Either way you're back to stock Mirasvit.

8. The relevance regression suite

`dev/symsearch/relevance-suite.csv` is a CSV of `query;expected_skus(pipe-separated);topN`. A row **passes** if **any** expected SKU appears in the top *N* results. Lines starting with `#` are comments.

Workflow: 1. `symsearch:relevance:run dev/symsearch/relevance-suite.csv --store 13 --suggest` to see current top SKUs per query. 2. Paste the SKUs that *should* anchor each query into column 2. 3. Run without `--suggest` to assert (non-zero exit on any failure) — wire it into CI or a pre-deploy check.

Anchor on **semantically meaningful** results (a telescope for "loves space", a Lebanese-cookbook for the French query) so a regression in semantic recall is caught even when keyword search would still return something generic.

9. Troubleshooting

Symptom	Likely cause & fix
Storefront search returns 0 results for everything after a reindex	Not this module per se — OpenSearch 3.3 + Mirasvit wildcards crash under concurrent segment search. The module's index plugin sets <code>concurrent_segment_search.mode=none</code> ; confirm it's applied on the live index (<code>.../_settings</code>). See TECHNICAL.md §9.
Semantic queries return keyword-quality results only	Storefront not enabled for that store view; or the query embedding is failing (check <code>var/log/system.log</code> for <code>[symsearch] query embedding failed</code>) — usually a bad/missing API key. Search still works (fallback), just without semantics.
<code>embed:status</code> shows lots of failed	Rate limit or network. Raise throttle, re-run <code>embed:generate</code> . Failures self-recover via retry + sweep.
A query 400s silently → empty results	Magento's OpenSearch adapter swallows query errors and logs <code>report.CRITICAL</code> in <code>var/log/exception.log</code> . Check there for the real reason.
Vectors not appearing in the index	Did you reindex <i>after</i> items went <code>ready</code> ? Reindex injects vectors; embedding alone doesn't.
Pipeline / plugin missing error from <code>pipeline:setup</code>	The OpenSearch instance lacks <code>opensearch-knn</code> / <code>opensearch-neural-search</code> . Install them.

Useful checks:

```
bin/magento symsearch:embed:status                # coverage
bin/magento symsearch:pipeline:setup                # plugins + pipeline
# vectors present in the live index (store 13 example):
curl -s "localhost:9200/magento2_product_13_v*/_count" -H 'Content-Type: application/json' \
  -d '{"query":{"exists":{"field":"embedding_vector"}}}'
```

10. Quick reference

- **Config root:** `symsearch/*` (Stores → Config → JALabs → Semantic Search)
- **Tables:** `jalabs_symsearch_item` (status), `jalabs_symsearch_vector` (vectors)
- **Queue:** topic/consumer `jalabs.symsearch.embed` / `jalabsSymsearchEmbed`
- **Pipeline:** `jalabs_symsearch_hybrid`

- **Index field:** `embedding_vector` (knn_vector)
- **Logs:** `var/log/system.log` (`[symsearch]` lines), `var/log/exception.log` (swallowed OpenSearch errors)
- **Crons (group `index`):** dispatch `*/5 * * * *`, sweep `0 3 * * *`